

ARQUITECTURA DE PLATAFORMA MULTI-EXCHANGE Y MULTI-BROKER

CCXT • MetaTrader 4 • MetaTrader 5 • cTrader • FIX • QuickFIX • OMS • EMS • Risk • Smart Routing

Versión: 2.0 (Master Document)	Año: 2026	Estado: Approved Specification	Ámbito: Core Architecture & Concurrency Blueprint
--------------------------------	-----------	--------------------------------	---

1. OBJETIVO DEL DOCUMENTO

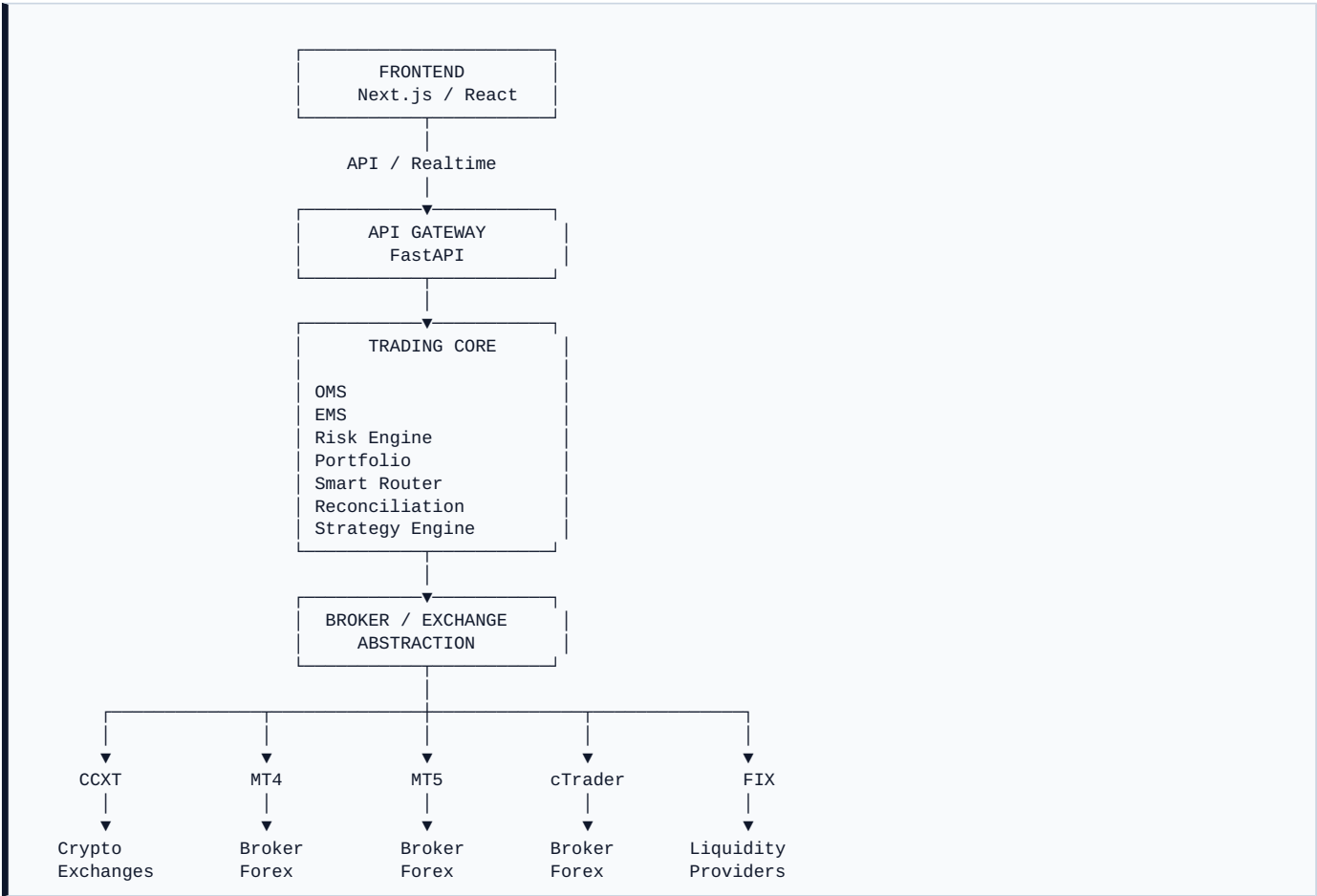
Diseñar una arquitectura profesional, modular y extensible para construir una plataforma de trading de grado institucional capaz de conectarse y operar concurrentemente con:

- **Exchanges de criptomonedas:** Spot, margen y derivados perpetuos vía REST y WebSockets (Binance, Bybit, OKX, Kraken, Coinbase, etc.).
- **Brokers Forex y CFD:** Conectividad a terminales y plataformas MetaTrader 4, MetaTrader 5 y cTrader.
- **FIX Protocol & Liquidity Providers:** Conexión directa con Prime Brokers y LPs institucionales mediante FIX 4.2, 4.4, 5.0 y FIXT 1.1.
- **APIs REST, WebSockets y Conectores Propietarios:** Puertos de entrada y salida desacoplados para protocolos heterogéneos.

Axioma Arquitectónico Central
«CCXT, MT4, MT5, cTrader y FIX deben considerarse conectores/infraestructura, no el núcleo de la aplicación.»
El núcleo debe ser propiedad absoluta del proyecto y permanecer 100% independiente de los proveedores externos.

2. VISIÓN GENERAL DE LA ARQUITECTURA

El flujo integral desacopla el cliente, la orquestación de negocio, los motores de decisión y la conectividad a mercados:



3. PRINCIPIO FUNDAMENTAL: AISLAMIENTO ESTRICTO

La aplicación **NO debe** hacer importaciones directas de librerías de conectividad o transporte en capas de negocio:

- Prohibido `import ccxt` desde cualquier parte del sistema salvo en `infrastructure/connectivity/ccxt`.
- Prohibido `import MetaTrader5` desde servicios de negocio o casos de uso.
- Prohibido `from ctrader_open_api import ...` desde el dominio.
- Prohibido depender directamente de `quickfix` desde los casos de uso.
- Todos esos componentes deben estar estrictamente aislados dentro de `infrastructure/`.

4. CAPAS DE LA PLATAFORMA

```
DOMAIN
↓
APPLICATION
↓
PORTS / INTERFACES
↓
INFRASTRUCTURE
↓
EXTERNAL PROVIDERS
```

Domain: Contiene Entidades y Value Objects como `Order`, `Position`, `Balance`, `Account`, `Market`, `Symbol`, `Ticker`, `OrderBook`, `Trade`, `Fill`, `Execution`, `Portfolio`, `Exposure`, `Fee` y `PnL`. No conoce CCXT, MT4, MT5, cTrader, FIX, Supabase, Redis ni FastAPI.

5. DOMAIN CORE: ENTIDADES Y VALUE OBJECTS

```
from dataclasses import dataclass
from decimal import Decimal
from datetime import datetime
from enum import Enum

@dataclass(frozen=True)
class Symbol:
    base: str
    quote: str

    @property
    def value(self) -> str:
        return f"{self.base.upper()}/{self.quote.upper()}"

    def __str__(self) -> str:
        return self.value

class OrderSide(Enum):
    BUY = "BUY"
    SELL = "SELL"

class OrderType(Enum):
    MARKET = "MARKET"
    LIMIT = "LIMIT"
    STOP = "STOP"
    STOP_LIMIT = "STOP_LIMIT"

class OrderStatus(Enum):
    PENDING = "PENDING"
    OPEN = "OPEN"
    PARTIALLY_FILLED = "PARTIALLY_FILLED"
    FILLED = "FILLED"
    CANCELLED = "CANCELLED"
    REJECTED = "REJECTED"
    EXPIRED = "EXPIRED"
    UNKNOWN = "UNKNOWN"

@dataclass
class Order:
    id: str
    client_order_id: str | None
    exchange: str
    account_id: str
    symbol: Symbol
    side: OrderSide
    type: OrderType
    amount: Decimal
    price: Decimal | None
    filled: Decimal
    remaining: Decimal
    status: OrderStatus
    timestamp: datetime

@dataclass
class Position:
    account_id: str
    symbol: Symbol
    quantity: Decimal
    entry_price: Decimal
    mark_price: Decimal
    unrealized_pnl: Decimal
    realized_pnl: Decimal
```

6. INTERFAZ UNIVERSAL DE BROKER (BROKER PORT)

```
from abc import ABC, abstractmethod

class Broker(ABC):

    @property
    @abstractmethod
    def id(self) -> str:
        pass

    @abstractmethod
    async def connect(self):
        pass

    @abstractmethod
    async def disconnect(self):
        pass

    @abstractmethod
    async def get_account(self):
        pass

    @abstractmethod
    async def get_balance(self):
        pass

    @abstractmethod
    async def get_positions(self):
        pass

    @abstractmethod
    async def get_orders(self):
        pass

    @abstractmethod
    async def get_order(self, order_id: str):
        pass

    @abstractmethod
    async def place_order(self, request):
        pass

    @abstractmethod
    async def cancel_order(self, order_id: str):
        pass

    @abstractmethod
    async def modify_order(self, order_id: str, request):
        pass
```

Esta interfaz representa únicamente las capacidades universales comunes. No se deben forzar particularidades de un conector concreto dentro de esta abstracción.

7. SEPARAR MARKET DATA Y TRADING

MarketDataProvider

```
class MarketDataProvider:
    async def ticker(self, symbol):
        pass

    async def orderbook(self, symbol):
        pass

    async def trades(self, symbol):
        pass
```

TradingProvider

```
class TradingProvider:
    async def get_balance(self):
        pass

    async def get_positions(self):
        pass

    async def place_order(self, request):
        pass

    async def cancel_order(self, order_id):
        pass
```

Permite admitir proveedores heterogéneos: trading sin libro L2, market data sin ejecución, feeds por streaming, FIX o REST.

8. ESTRUCTURA DE ADAPTADORES DE CONECTIVIDAD

```
infrastructure/
├── connectivity/
│   ├── ccxt/
│   │   ├── adapter.py
│   │   ├── mapper.py
│   │   └── factory.py
│   ├── mt4/
│   │   ├── adapter.py
│   │   ├── bridge.py
│   │   └── mapper.py
│   ├── mt5/
│   │   ├── adapter.py
│   │   ├── bridge.py
│   │   └── mapper.py
│   ├── ctrader/
│   │   ├── adapter.py
│   │   ├── client.py
│   │   └── mapper.py
│   └── fix/
│       ├── adapter.py
│       ├── session.py
│       ├── mapper.py
│       └── client.py
```

9. CCXT (CAPA DE INTEGRACIÓN CRYPTO)

CCXT se utiliza exclusivamente para comunicar con exchanges compatibles (Binance, Bybit, OKX, Kraken, Coinbase). El resto del sistema no conoce CCXT.

- Conversión estricta de símbolos hacia y desde la nomenclatura de cada exchange.
- Normalización de órdenes, balances, posiciones y estados de ejecución.
- Gestión determinista de rate limits, reintentos exponenciales y mapeo de excepciones nativas.

10. METATRADER 5 (MT5 BRIDGE & TERMINAL)

El terminal MT5 requiere ejecución en entorno Windows y no se comporta como una API REST clásica.

```
Trading Core → MT5Adapter → MT5 Bridge / Gateway → MetaTrader 5 Terminal (Windows) → Broker
```

Evaluación de Proyectos Open-Source de Referencia:

- [PTHAICAP/mt5-python-bridge](#) : Puente basado en ZeroMQ/WebSockets para comunicación de procesos. Buena latencia pero requiere control de reconexión y serialización de mensajes M2M.
- [monki103/pyMt5Bridge](#) : Servidor HTTP REST (Flask/FastAPI) que desacopla el core Linux de la terminal Windows. Arquitectura limpia y estable para despliegue desacoplado.
- [elitekaycy/mt5-gateway](#) : Gateway estructurado para exponer endpoints de órdenes y streaming. Requiere control estricto de timeouts transaccionales.

Aspectos clave: Sincronización continua, mapeo de cuentas Netting vs Hedging, normalización de sufijos de broker (ej. [EURUSD.pro](#), [EURUSDm](#)) y filling modes (FOK, IOC, Return).

11. METATRADER 4 (MT4 BRIDGE)

MT4 y MT5 **no son equivalentes**. MT4 no ofrece API nativa oficial en Python y requiere mecanismos específicos:

```
Trading Core → MT4Adapter → MT4 Bridge / Auxiliary Process → MT4 EA (MQL4) → MetaTrader 4 → Broker
```

El adaptador considera integración mediante Expert Advisors (EAs) que abren sockets TCP/ZeroMQ locales, manejo de tickets específicos y control de requotes/slippage mediante el Reconciliation Service.

12. CTRADER (SPOTWARE OPEN API)

Integración basada en cTrader Open API mediante mensajes Google Protocol Buffers (Protobuf) sobre TCP/TLS.

Trading Core → cTrader Adapter → cTrader Open API (Protobuf TLS) → cTrader → Broker

Evaluación de [spotware/OpenApiPy](#) :

- Librería oficial de referencia de Spotware para autenticación y streaming.
- *Consideración técnica:* OpenApiPy se basa en Twisted. En una arquitectura orientada a [asyncio](#) , se debe utilizar el reactor asíncrono o un worker desacoplado para no bloquear el bucle de eventos principal.
- Soporte nativo para streaming reactivo de órdenes, posiciones, cuentas y rate limits.

13. FIX PROTOCOL & 14. QUICKFIX ENGINE

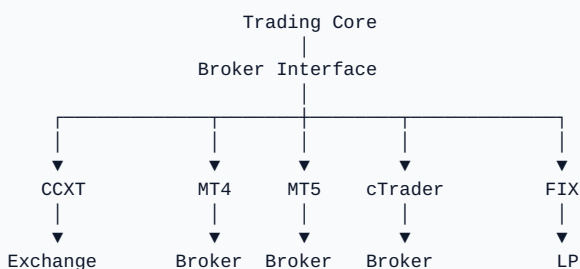
FIX Protocol es la categoría institucional para conexión directa con Liquidity Providers (LPs), Prime Brokers y ECNs.

Trading Core → FIX Adapter → QuickFIX Engine → FIX Session (TLS) → Liquidity Provider

Evaluación de QuickFIX ([quickfix/quickfix](#)):

- Soporte de especificaciones FIX 4.2, 4.4, 5.0 y FIXT 1.1.
- Gestión robusta de sesiones: Logon (35=A), Heartbeat (35=0), Sequence Numbers, Resend Requests (35=2) y Execution Reports (35=8).
- Persistencia de mensajes y recuperación de secuencias tras desconexión sin pérdida transaccional.
- **QuickFIX permanece estrictamente dentro de Infrastructure:** ningún puntero o clase C++ se propaga al dominio.

15. ARQUITECTURA GLOBAL DE CONECTIVIDAD



16. OMS — ORDER MANAGEMENT SYSTEM

Responsabilidades del OMS: recibir órdenes, generar [client_order_id](#) único, validar reglas, consultar al Risk Engine, seleccionar cuenta, enviar a ejecución, persistir en base de datos, actualizar estados, gestionar fills, detectar timeouts y reconciliar.

Client → OMS → Risk Engine → Router → Execution → Broker Adapter

17. EMS — EXECUTION MANAGEMENT SYSTEM

Separación conceptual estricta:

- **OMS:** «¿Qué quiere hacer el usuario?» → Registra la intención comercial: [BUY 10 BTC](#) o [BUY 10 Lotes EURUSD](#) .
- **EMS:** «¿Cómo y dónde se ejecuta?» → Divide y fragmenta:
 - Cripto: Exchange A → 4 BTC | Exchange B → 3 BTC | Exchange C → 3 BTC.
 - Forex/CFD: Broker A (MT5) → 3 Lotes | Broker B (cTrader) → 3 Lotes | Broker C (FIX) → 4 Lotes.

18. SMART ORDER ROUTER (SOR) & EFFECTIVE EXECUTION COST

El router no elige simplemente el precio más bajo visible (lowest ask). Calcula el **Coste Efectivo de Ejecución** evaluando:

Precio en el libro • Spread • Profundidad L2 • Liquidez disponible • Comisiones Maker/Taker • Slippage proyectado • Latencia de red estimada • Probabilidad de fill • Balance libre • Exposición y límites de riesgo • Estado del conector.

19. PRE-TRADE RISK ENGINE

El Risk Engine se ejecuta de forma síncrona en memoria antes de enviar cualquier orden:



Controles: Tamaño máximo, exposición máxima total/por broker/por símbolo/por estrategia, balance mínimo, pérdida diaria acumulada, pérdida máxima (drawdown), límite de velocidad de órdenes, circuit breakers, kill switch global y deduplicación estricta de órdenes.

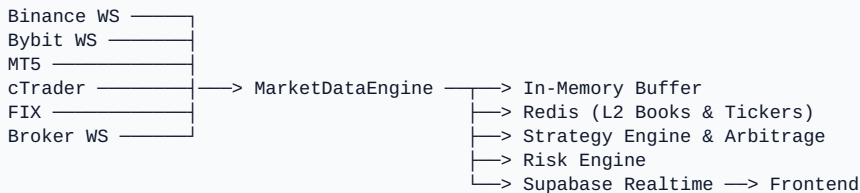
20. RECONCILIATION SERVICE

La reconciliación continua es obligatoria para neutralizar desincronizaciones por timeouts, caídas de red, fills parciales, cancelaciones externas o reinicios de procesos:



Principio de Verdad Absoluta: El estado reportado por el broker/exchange es siempre la fuente final incuestionable de verdad.

21. MARKET DATA ENGINE & 22. STREAMING VS REST



Canal	Responsabilidad	Regla de Diseño
REST	Balances, histórico de órdenes, cuentas, catálogo de mercados, reconciliación.	Peticiones puntuales no bloqueantes.
STREAMING	Tickers, order books, trades, fills, order updates, eventos de cuenta.	Nunca diseñar la arquitectura alrededor de polling continuo.

23. REDIS & 24. SUPABASE

Redis (Hot State): Almacena últimos precios, libros L2 volátiles, estado de cuentas en memoria, distributed locks (Redlock), idempotency keys y coordinación temporal. *No es la fuente definitiva de persistencia.*

Supabase (PostgreSQL + Auth + Realtime): Base de datos relacional transaccional, autenticación JWT, Row Level Security (RLS) y persistencia auditada. *Supabase Realtime es el canal downstream hacia el Frontend; NO debe usarse como motor de ingesta de sockets financieros.*

25. BASE DE DATOS (ESQUEMA RELACIONAL)

Estructura inicial de tablas en PostgreSQL:

users • accounts • brokers • exchange_accounts • markets • orders • order_events • fills • positions • balances • portfolio_snapshots • executions • fees • strategies • strategy_runs • alerts • audit_logs

Posteriormente ampliable a:
liquidity_providers • fix_sessions • broker_connections • reconciliation_runs • risk_events • routing_decisions

26. SEGURIDAD

- Nunca enviar credenciales, API secrets o contraseñas al frontend.
- Separar estrictamente la configuración pública de los secretos (KMS / Vault / variables de entorno).
- Cifrado AES-256-GCM para llaves de trading; permisos de retiro (withdrawals) estrictamente deshabilitados.

- Row Level Security (RLS) activado en todas las tablas transaccionales de Supabase.

27. ABSTRACCIÓN DE CUENTAS (ACCOUNT ABSTRACTION)

User → TradingAccount → BrokerConnection

Un usuario puede tener:

```
User
├── Binance Account (Spot)
├── Binance Account 2 (Futures)
├── MT5 Account
├── cTrader Account
└── FIX Account
```

28. NORMALIZACIÓN DE SÍMBOLOS

El dominio define `Symbol(base="EUR", quote="USD")`. Cada adaptador convierte bidireccionalmente entre el símbolo canónico y el símbolo nativo del broker (ej. `EURUSD`, `EURUSDm`, `EURUSD.pro`, `BTC/USDT`, `BTCUSDT.P`).

29. NORMALIZACIÓN DE ÓRDENES & 30. ESTADOS DE ORDEN

Enums de dominio (`OrderType`: `MARKET`, `LIMIT`, `STOP`, `STOP_LIMIT`; `OrderStatus`: `PENDING`, `OPEN`, `PARTIALLY_FILLED`, `FILLED`, `CANCELLED`, `REJECTED`, `EXPIRED`, `UNKNOWN`). El adaptador mantiene además el estado crudo nativo para depuración.

31. EVENT BUS ASÍNCRONO

Eventos internos: `OrderCreated`, `OrderAccepted`, `OrderRejected`, `OrderPartiallyFilled`, `OrderFilled`, `OrderCancelled`, `PositionUpdated`, `BalanceUpdated`, `TickerUpdated`, `MarketDisconnected`, `BrokerDisconnected`, `RiskLimitTriggered`.

```
OrderFilled ──> PortfolioService (Actualización de inventario)
               ├── PnLService (Cálculo de resultado)
               ├── RiskEngine (Márgenes)
               ├── ReconciliationService
               ├── Supabase Database (Persistencia)
               └── Supabase Realtime → Frontend
```

32. OBSERVABILIDAD

Structured logging en formato JSON, tracing distribuido (OpenTelemetry), métricas de latencia de red, tiempo de ejecución de órdenes, tasa de rechazo y **Broker Health Dashboard** en vivo:

```
Broker Health:
├── MT5      ONLINE (Latency: 14ms)
├── MT4      ONLINE (Latency: 28ms)
├── cTrader  ONLINE (Latency: 22ms)
├── FIX      ONLINE (Latency: 4ms)
└── CCXT     ONLINE (Latency: 35ms)
```

33. GESTIÓN UNIVERSAL DE ERRORES

```
class TradingError(Exception):
    pass

class ConnectionError(TradingError):
    pass

class OrderRejectedError(TradingError):
    pass

class RateLimitError(TradingError):
    pass

class InsufficientBalanceError(TradingError):
    pass

class ReconciliationError(TradingError):
    pass
```


Todos los errores específicos de CCXT, MT5, MT4, cTrader y FIX se traducen obligatoriamente a esta jerarquía canónica.

34. IDEMPOTENCIA TRANSACCIONAL

```
Request (client_order_id) → Idempotency Check (Redis NX) —┬─ Already exists → Return existing order
                                                         └─ New → Execute & Persist
```

Garantiza que una orden nunca se envíe dos veces por reintentos de red, timeouts o reconexiones de workers.

35. ARQUITECTURA DE PROCESOS

Comenzar como un **monolito modular asíncrono** y desacoplar en producción hacia servicios independientes: API Service, Market Data Service, Execution Service (EMS), Risk Service, Reconciliation Service, Strategy Service, Worker Service y Realtime Publisher.

36. ESTRUCTURA INTEGRAL DEL REPOSITORIO

```
trading-platform/
├── app/
│   ├── domain/
│   │   ├── entities/
│   │   ├── value_objects/
│   │   ├── enums/
│   │   └── events/
│   ├── application/
│   │   ├── services/
│   │   └── use_cases/
│   ├── ports/
│   │   ├── broker.py
│   │   ├── market_data.py
│   │   ├── execution.py
│   │   ├── repository.py
│   │   └── event_bus.py
│   ├── infrastructure/
│   │   ├── connectivity/
│   │   │   ├── ccxt/
│   │   │   ├── mt4/
│   │   │   ├── mt5/
│   │   │   ├── ctrader/
│   │   │   └── fix/
│   │   ├── persistence/
│   │   │   └── supabase/
│   │   ├── cache/
│   │   │   └── redis/
│   │   └── messaging/
│   ├── api/
│   │   ├── routes/
│   │   └── dependencies.py
│   └── main.py
├── tests/
│   ├── domain/
│   ├── application/
│   ├── connectivity/
│   └── integration/
├── migrations/
├── docs/
├── scripts/
├── .env
├── pyproject.toml
└── README.md
```

37. TECNOLOGÍAS PROPUESTAS & 38. PROYECTOS OPEN-SOURCE ANALIZADOS

- **Backend:** Python 3.12, FastAPI, asyncio, Pydantic v2, pytest.
- **Connectivity:** CCXT, MT4 Socket Bridge, MT5 Windows Gateway, cTrader OpenApiPy, QuickFIX.
- **Infrastructure & DB:** Supabase (PostgreSQL 16, Auth, Realtime), Redis 7.
- **Frontend:** Next.js 14+, React, TypeScript.

- **Proyectos Evaluados:** [ccxt/ccxt](#) (transporte crypto), [PTHAICAP/mt5-python-bridge](#), [monki103/pyMt5Bridge](#) y [elitekaycy/mt5-gateway](#) (desacoplamiento de MT5 en Windows), [spotware/OpenApiPy](#) (Twisted/Protobuf cTrader), [quickfix/quickfix](#) (sesión y secuencias FIX).